

MODULE 5.1 · Concept 2 of 7

# Thought-Action- Observation Loop

*Deep dive into the four-step ReAct cycle with a complete worked example*

**9 min**

Duration

**B3 – Apply**

Bloom's

**L2 – Intermediate**

Level

**TH-000001**

Prerequisite

AI-AG-IN-TH-000002

Prof. Sashikumaar Ganesan · IISc Bangalore · AhamX Bodhi

## SECTION 1 — THE FOUR STEPS

**Thought → Action → Observation →  
Repeat**

*Each step has a precise role in the agent's reasoning process*

# The Four Steps Defined

Every ReAct iteration follows this exact sequence

- 1 Thought** ✓ The model reasons ALOUD about what it knows, what it needs, and what to do next. This is Chain-of-Thought inside the agent loop. The thought is generated as tokens — it's visible in the trace.
- 2 Action** ✓ The model issues a STRUCTURED tool call — a JSON object with tool name and parameters. This is NOT free text. The orchestrator parses this and executes the real tool.
- 3 Observation** ✓ The tool returns a result that is INJECTED into the context window. The model did NOT generate this — it came from the real world. This is what grounds the reasoning.
- 4 Repeat or Answer** The model reads the observation and DECIDES: do I have enough information to answer, or do I need another Thought→Action→Observation cycle? If enough → generate final Answer.

= Ground Truth

# Thought: The Model Reasons Aloud

*What happens inside a Thought step — and why it matters*

## Example Thoughts from a Real Agent Trace:

**Thought 1:** "The user wants the Nifty 50 change today. I need two data points: today's close and yesterday's close. Let me search for today's value first."

**Thought 2:** "I have today's close (22,474) from the search. I also know yesterday's close was 22,310 from the same result. Now I can calculate the percentage change."

**Thought 3:** "The calculation shows +0.74% I now have enough information to give the final answer."

[T1] → web-search()

[T2] → Calculator()

## A Thought does three things:

1. Assesses current state: 'What do I know? What do I still need?'
2. Plans next action: 'I should search / calculate / look up...'
3. Evaluates progress: 'Am I done? Do I need more steps?'

# Action: Structured Tool Call

*The model generates a parseable JSON tool call — not free text*

## What the Model Generates:

```
Action: web_search(  
  query="Nifty 50 close price today"  
)
```

--- or in JSON format ---

```
{  
  "tool": "web_search",  
  "args": {  
    "query": "Nifty 50 close today"  
  }  
}
```

## What the Orchestrator Does:

1. Parse the JSON tool call
2. Validate tool name exists in registry
3. Validate parameters match schema
4. Execute the real API/function
5. Capture the result string
6. Inject result as 'Observation' into context

The model NEVER executes tools directly.  
Your code is always the intermediary.

# Observation: Real Data Enters the Context

*The tool result grounds the agent's next reasoning step*

**The Observation is the ONLY part of the loop that comes from the real world:**

| Tool Called    | Observation Returned                                                                                            | Format                          |
|----------------|-----------------------------------------------------------------------------------------------------------------|---------------------------------|
| web_search     | {"results": [{"title": "Nifty 50 closes at 22,474...", "snippet": "...up 164 points from yesterday's 22,310"}]} | Structured JSON from search API |
| calculator     | "(22474 - 22310) / 22310 * 100 = 0.7349..."                                                                     | Exact numerical result          |
| code_executor  | "stdout: [2,3,5,7,11,13,17,19,23,29]\nreturn_code: 0"                                                           | Program output + exit code      |
| database_query | [{"name": "TCS", "revenue": "₹2.41T"}, {"name": "Infosys", "revenue": "₹1.62T"}]                                | SQL result set as JSON          |

Yag\_Search()

***Observations are NOT generated by the model. They are injected by the orchestrator. This is what prevents hallucination.***

# Repeat or Answer: The Decision Point

*The model decides: continue the loop or generate the final answer*

## When to ANSWER (stop looping):

- All required information has been gathered
- The goal has been achieved (e.g., booking confirmed)
- The calculation/synthesis is complete
- Max iterations reached (forced stop)

Signal: 'I now have enough information.'

## When to REPEAT (continue looping):

- Missing data still needed for the answer
- Previous action failed (need to retry/reformulate)
- Result was partial — need additional lookups
- Need to verify/cross-check information

Signal: 'I still need to find/check...'

SECTION 2 — WORKED EXAMPLE

# Nifty 50 Change: Full 7-Step Trace

*Following every Thought, Action, and Observation*



# Nifty 50 Trace: Steps 1-3

*Search for today's index value*

## Thought 1

*reasoning*

"I need today's Nifty 50 close price. Let me search for it."

## Action 1

*tool call*

```
web_search(query="Nifty 50 close price today February 2026")
```

## Observation 1

*tool result*

"Nifty 50 closed at 22,474.05 on Feb 23, 2026, up 164 points from previous close of 22,310.35"

*Notice: the Observation contains BOTH data points we need (today's close AND yesterday's). A good search query saved an extra iteration.*

# Nifty 50 Trace: Steps 4-7

*Calculate the change and deliver the answer*

## Thought 2

"I have both values: today=22,474.05, yesterday=22,310.35. Let me calculate the % change."

## Action 2

```
calculator(expression="(22474.05 - 22310.35) / 22310.35 * 100")
```

## Observation 2

"0.7339..."

## Thought 3

"The change is +0.73%. I have all information needed to answer."

## Answer

"The Nifty 50 rose by approximately 0.73% today, closing at 22,474 (up 164 points from 22,310)."

**Total:** 7 steps, 2 tool calls, ~920 tokens, ~\$0.0006 at GPT-4o-mini pricing. A human would take 2 minutes for the same task.

# Token Map: What Each Step Generates

*Understanding where tokens come from helps optimize cost*

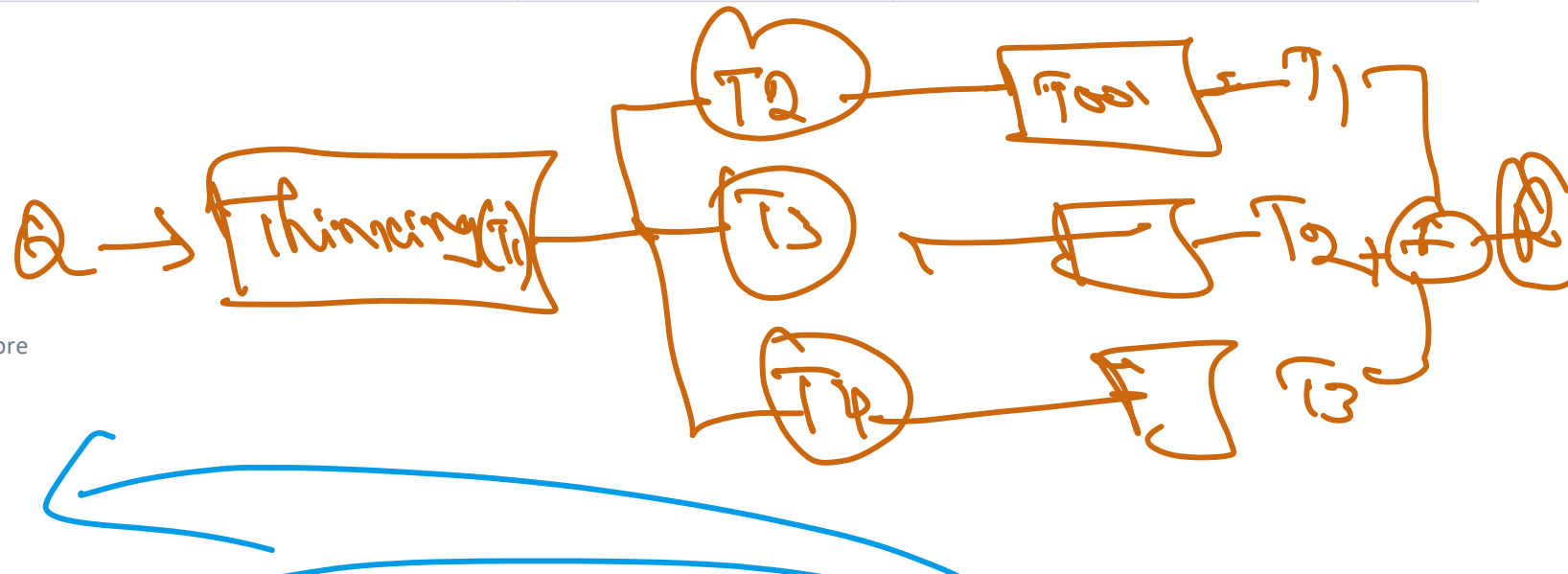
| Step Type     | Generated By        | Token Type      | Typical Size   | Cost Driver                |
|---------------|---------------------|-----------------|----------------|----------------------------|
| Thought       | LLM (output tokens) | Generated text  | 30–60 tokens   | Output pricing             |
| Action        | LLM (output tokens) | Structured JSON | 20–50 tokens   | Output pricing             |
| Observation   | Tool (injected)     | Input context   | 50–200 tokens  | Input pricing              |
| Answer        | LLM (output tokens) | Generated text  | 50–150 tokens  | Output pricing             |
| System prompt | Developer (static)  | Input context   | 300–500 tokens | Input pricing (every call) |

*Key optimization: Observations (tool results) are the largest variable cost. Truncate long results to relevant sections.*

# Multi-Step vs. Single-Step Tasks

Some tasks need 1 iteration, others need 5+ — how to tell

| Task Type                 | Example                                | Iterations                        | Why                                      |
|---------------------------|----------------------------------------|-----------------------------------|------------------------------------------|
| Single-step lookup        | "What is the capital of France?"       | 0 (direct answer)                 | LLM already knows — no tools needed      |
| Single-tool lookup        | "What is AAPL stock price?"            | 1 (search + answer)               | One search provides the answer           |
| Multi-tool sequential     | "Nifty 50 % change today?"             | 2–3 (search + calc)               | Need to chain results between tools      |
| Multi-tool with branching | "Compare TCS vs Infosys revenue"       | 3–5 (2 searches + compare)        | Multiple independent lookups + synthesis |
| Complex research          | "Write a market analysis of Indian IT" | 5–10+ (many searches + synthesis) | Open-ended, needs multiple sources       |



# Reading and Debugging ReAct Traces

*What to look for when an agent gives a wrong answer*

## Check the final Answer

Does it match the user's question? Is it grounded in Observations?

Answer includes information NOT in any Observation = hallucination

## Check each Observation

Did the tool return relevant data? Was it parsed correctly?

Observation is empty, error, or irrelevant = bad search query

## Check each Action

Was the right tool called? Were parameters correct?

Wrong tool or bad parameters = reasoning failure

## Check each Thought

Does the reasoning logically lead to the next Action?

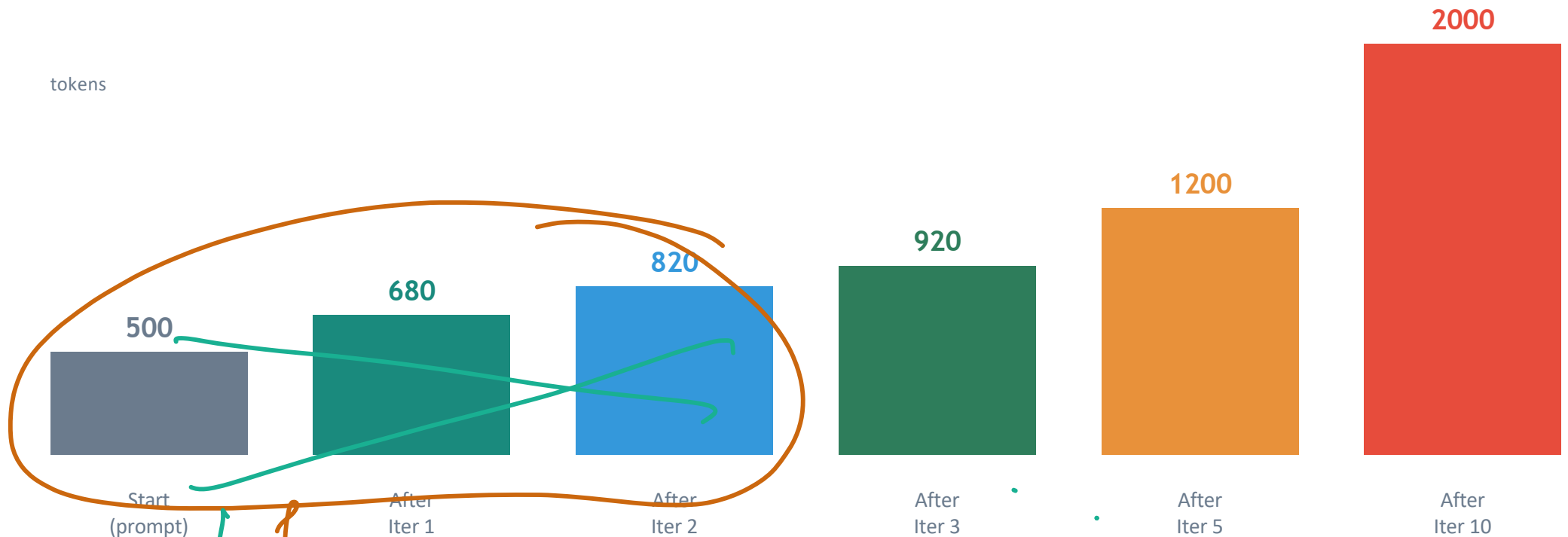
Thought says one thing, Action does another = CoT breakdown

**Pro tip: debug backwards. Start from the wrong answer, trace back through Observations to find where the data went wrong.**

# Context Window Growth Across Iterations

Each iteration adds ~120–200 tokens — budget accordingly

tokens



At 10 iterations, context approaches 2K tokens. With a 4K-token model, you'd have only 2K left for the system prompt + answer.  
Budget rule:  $\text{max\_iterations} \times \sim 200 \text{ tokens/iter}$  should not exceed 50% of your model's context window.

Prof. Sashikumaar Ganesan | IISc Bangalore

Compress(?)

~ 1000 tokens  
1200



# Key Takeaways

*The essential points from this concept*

**Four steps per iteration:** Thought (reason aloud), Action (tool call), Observation (result), Repeat or Answer.

**Thoughts make actions intentional;** Observations make thoughts grounded. Both are logged.

**A 3-iteration trace uses ~920 tokens** (~\$0.0006 at GPT-4o-mini) — cheaper than a cup of chai.

**Debug backwards:** start from wrong answer, trace through Observations to find the root cause.

**Budget context growth:** ~200 tokens/iteration. Don't let iterations consume >50% of the window.

**Next:** AI-AG-IN-TH-000003 — Tool Use and Function Calling

AI-AG-IN-TH-000002 Complete

# Thought-Action- Observation Loop

Concept 2 of 8

Advanced Certification in Agentic & Generative AI · IISc Bangalore

Prof. Sashikumaar Ganesan · AhamX Bodhi